

FAIR-Bench: Evaluating how well video generation models understands physics using intervention robustness

Project Overview

Video generators are increasingly being framed as world models, with the idea that if the model can predict plausible future frames, then it must know how the physical world behaves. This is controversial, since some say it's just memorizing statistics.

“Video prediction is increasingly viewed as a path toward generalizable world models, yet it remains unclear whether these systems learn underlying causal structure or merely exploit superficial visual correlations for future prediction.” - CRONOS

CRONOS is a past benchmark that attempts to measure how well models understand physics. However the metrics used seem to be flawed(See appendix)

So building on top of that, we develop our own benchmark, attempting to better-measure the physics understanding of video models.

FAIR stands for *Framework for Assessing Intervention Robustness*, which is a conceptual framework used in our benchmark(inspired by CRONOS)

Research Question

High level: How well does video generation models have internal physics understanding?

Specific:

1. Are the models robust to interventions? (Main metric, where FAIR comes from)
 - Given two different video starting frames of an object moving that only differ in the object's color, how similar are the video models' generations in the tracked ball trajectories?
 - Given two different video starting frames of an object moving that only differ in the background, how similar are the video models' generations in the tracked ball trajectories?
 - Given three different video starting frames of an object moving that only differ in the camera angle, how similar are the video models' generations in the View-Invariant Motion Similarity (VIMS) metric, which compares the three generations at once by triangulating the ball (see Appendix: VIMS)?
2. Are the videos generated by the models physically plausible? (Secondary metric)
 - Physics-informed scores from Morpheus on the tracked ball trajectories (do they follow the equations of motion and conservation laws?)
3. How similar are the generated videos to the reference video, specifically the simulator's continuation? (Third metric)
 - Reference error on the tracked ball trajectories (distance from the ball in the reference video, frame by frame)

Whole Pipeline

1. Choose models to evaluate(*No need to be the same selection with CRONOS, we're not doing ablation study with that paper)

- See Appendix: Models&Cost for model options and cost
- V2V only

2. Measure intervention robustness of the models

1. Generate dataset using Kubric

- #1 Ball thrown
- #2 Ball falling and bouncing
- #3 Ball occluding(disappear and reappear)
- All scenes have a determined future: everything the rest of the video depends on (e.g., the first bounce, the ball's speed before it goes behind the occluder) is visible in the first 2 seconds, so only one continuation is physically correct
- For each of the categories, create interventions as in the table below: color, background and camera angle for #1; color and background for #2; color for #3

Motion type	Ball thrown									Ball falling and bouncing						Ball occluding		
Interventions	color			bg			angle			color			bg			color		
	r	g	b	1	2	3	1	2	3	r	g	b	1	2	3	r	g	b

- So total of 18 generations per model
- Rendered with Kubric (PyBullet physics, Blender Cycles): 1280x720, 24 fps, 6 s per clip. The first 2 s are the model input; the other 4 s are the simulator's continuation (the reference video)
- Defaults: red ball, grey back wall, front camera. The background and camera-angle groups use the red ball. Backgrounds change only the back wall (grey, blue, near black, brick), so the floor the ball touches stays the same. The camera angles are three oblique views (48° left, 48° right, 52° from above), none of them the front camera
- Determined future: the thrown and bouncing balls hit the floor twice inside the input (thrown at 0.70 and 1.48 s, bouncing at 0.58 and 1.56 s). In the occluding scene the ball rolls at a constant 0.8 m/s, is fully visible during the input, starts to go behind the box at 2.5 s and is fully visible again from 4.4 s. Air drag and rolling friction are off
- Checks: all clips of a motion type share exactly the same 3D trajectory, and the rendered ball matches the ground-truth position within 0.6 px. On the three camera-angle clips the tracker error is about 0.1 px and VIMS gives E = 0.25 mm

2. Feed first 2 seconds of the videos (v2v) into the models and generate

- specific generation settings(prompt, random seeds, how many times we'll generate, etc) have to be defined and kept consistent for the same model
- n models x 18 videos

3. Calculate result for each intervention group(motion type x intervention type = 6 groups; for example ball thrown + color is one intervention group)

- metric for intervention by color and background: track the ball in each generation and compare the trajectories directly (the camera is the same, so ball positions are compared frame by frame)
 - trajectory distance compares by two only, so initially we get 3 numbers per intervention group: (red, green), (red, blue), (blue, green). use average of those three
- metric for intervention by camera angle: View-Invariant Motion Similarity (VIMS), which compares the three angles at once by triangulating the ball in every frame and measuring how far the three lines of sight are from meeting at one point
 - our own metric, see Appendix: VIMS for equation

4. Aggregate result

- we now have 6 scores(one for each intervention group) for each model
- Aggregate these to one score (decide after seeing the 6 scores)

3. Measure physical plausibility of the videos generated by the models

- Reuse the 18 generations per model in Step 2
- Borrow the method of Morpheus (arXiv 2504.02918): score the ball trajectories tracked in Step 2 with its two physics-informed scores. A generation only has to be physically possible; it is not compared with the simulator's continuation
 - Dynamical score: how well a physics-informed neural network that obeys the scene's equations of motion (e.g., constant gravitational acceleration in flight) can fit the trajectory
 - Physical Invariance score: how constant the quantities the scene must conserve (e.g., energy between bounces, horizontal momentum in flight) stay over time

4. Measure similarity to the reference video

- Reuse the 18 generations per model in Step 2
- Compare the tracked ball with the ball in the reference video frame by frame, in meters at the ball's depth (pixel error $\times$ true depth $\div$ focal length). For the camera angle group, use the distance from the true 3D position to each view's line of sight

5. Aggregate the three scores to one final number

- Determine ratio after seeing the scores

Expected Deliverables

- Paper, video dataset, code, metric

Appendix: CRONOS Motivation/Background

CRONOS benchmark evaluates video models' physics understanding with two metrics:

1. Success rate, measuring the physical plausibility of the videos generated
2. Sensitivity, measuring the counterfactual consistency of the video models
 - Counterfactual consistency is the model's generations being 'consistent' across 'counterfactual' contexts. 'Counterfactual' in this setting, can be thought of as intervention; only one factual aspect of the videos differ. For example, video clips of a red ball falling, and a blue ball falling, with all aspects being the same except for the color, these two video clips can be claimed as counterfactual contexts.
 - An oversimplified example of the measuring process looks like this:
 - Take the setting of a ball bouncing off the table. They first create reference videos with Unreal Engine. Two videos of a ball bouncing off the table, with the only difference being the color of the ball. Then they feed the first five frames of each reference video into the video generator model, and make it generate the rest of the video. So we have two reference videos and two generated videos. We now score the generated videos in five dimensions, which should collectively represent the physical plausibility of the generated videos. Now sensitivity is calculated with the difference of these five scores across the generations.
 - In the real setting there are more than two interventions(e.g. red, blue, green, ...) and more than one generation per reference, so they use averages and other equations, but this captures the core idea.

Problem

Long story short, two metrics are both flawed due to a score called 'motion similarity' that the metrics heavily depend on.

- Motion similarity: Among the five dimensions there is a score called 'motion similarity'. It evaluates how similar the motion of the given two videos are. The authors calculate this with the 'reference video' and the 'generated video'. However the generated video is unlikely to exhibit the similar motion as the reference video even if the generated video is physically plausible, since the model just 'sees' the starting frames, and doesn't know, for example, the elasticity of the ball. So it can produce so many possibilities of physically plausible videos, and the probability of the one being generated being similar to the reference video, is low. So physically plausible generations can be scored low in the motion similarity metric. This problem comes from the starting frames not determining the future. When they do (for example, the input already shows the ball bouncing), only one continuation is physically correct and comparing with the reference video is fair, which is how our scenes are built.

Also there's another problem on its own with sensitivity. To calculate sensitivity the authors compare the 'physical plausibility' of generations. It's saying that: if we change some aspect

about the video and the generation's physical plausibility changes at a minimum, then it's counterfactual consistent.

This is a questionable design. If the red ball generation was bouncing to the left and when we change the color to blue, and now the generation bounces to the right, it is not counterfactual consistent. However this sensitivity metric can define it as counterfactual consistent if both are very physically plausible. Two very different videos can be similar in physical plausibility.

What we're trying to do

We develop a new benchmark that properly measures physical plausibility, sensitivity and similarity to the reference video.

For physical plausibility, we borrow the physics-informed scores of Morpheus (arXiv 2504.02918) and compute them on the tracked ball trajectories. A generation only has to be physically possible; it is not compared with a reference video.

For sensitivity, we compare different generations directly with each other rather than with references: tracked ball trajectories for color and background, and VIMS for camera angle.

- FAIR stands for *Framework for Assessing Intervention Robustness*, since we calculate how 'robust' models are to 'interventions'.

For similarity to the reference video, we compare the tracked ball with the simulator's ball, frame by frame.

We then aggregate these three metrics to one number (hence the benchmark score)

This is framed as a new benchmark instead of something like CRONOS v2, we can put the specific CRONOS related motivations in the appendix section

Appendix: Models&Cost

- 720p, no audio
- Output length is 4s. Where a model's minimum output is longer than 4 s, the cost uses that minimum.
- I2V input is the last frame of the 2s clip. V2V input is the full 2s clip.
- The cost column uses the official price (or the model's main provider), named in parentheses. Notably cheaper resellers are listed in the last column.
- A model's future is only determined if the frames it conditions on include the last bounce (about 1.5 s). Veo 3.1 extension continues from the last 1 s of the input, which includes it; Cosmos 3 Nano takes at most 5 input frames (0.2 s), which do not

V2V Options

Model	Params	Cost / generation	Cost × 18	Where available (API)
xAI grok-imagin e-video	Undisclosed	\$0.30 (xAI: \$0.07/s × 4 + input \$0.01/s × 2)	\$5.40	xAI official · fal (input 2–15 s) · Replicate (\$0.05/s) · WaveSpeed (6 s / 10 s outputs only) · Atlas [unverified]
Gemini Omni 1.1 Flash	Undisclosed	~ 0.42 (<i>GeminiAPI</i> : ~ 0.10/s × 4 + input ~\$0.009/s × 2; audio always on)	~\$7.60	Gemini API (input ≤10 s; not available in EEA/UK/CH; GA since 8/27; the preview ended 9/30) · Vertex [preview] (input 1–30 s) · AIML (\$0.13/s) · Atlas · EvoLink [price unverified] · Kie [unverified]
Seedance 2.5	Undisclosed	\$0.97 (BytePlus; minimum-token rule applies)	\$17.42	BytePlus official (extend mode) · Volcengine (CN) · WaveSpeed (1.32) · <i>EvoLink</i> (1.08) · PiAPI · Runway (\$1.50) · fal / Replicate / OpenRouter / Kie / Segmind (reference-video route)
Seedance 2.0	Undisclosed	\$0.65 (BytePlus)	\$11.70	BytePlus · Volcengine · fal / Replicate / OpenRouter / Kie / Segmind / EvoLink / PiAPI (reference-video route) · WaveSpeed extend [not usable: last frame only]
Seedance 2.0 Fast	Undisclosed	\$0.50 (BytePlus; \$0.37 enterprise promo until 10/7)	\$8.98	Same as 2.0 · Segmind cheapest (~\$0.21)
Seedance 2.0 Mini	Undisclosed	\$0.32 (BytePlus; \$0.13 enterprise promo until 10/7)	\$5.72	Same as 2.0 · Segmind cheapest (~\$0.14)
MiniMax H3-Max	Undisclosed	\$0.40 (fal: \$0.08/s, 5 s minimum)	\$7.20	fal extend-video only (input 1.6–60 s)

Wan 3.0	Undisclosed [beta]	\$0.60 (Alibaba: input + output billed, \$0.10/s × 6)	\$10.80	Alibaba Model Studio (prompt-driven extend) · WaveSpeed extend (\$0.40, only new frames billed; Prime \$0.15/s)
Wan 2.7	Undisclosed	\$0.60 (Alibaba: billed on total length incl. input, \$0.10/s × 6)	\$10.80	Alibaba (first-clip continuation mode) · fal · Replicate · Atlas · Novita · WaveSpeed extend
Wan 2.6	Undisclosed	\$0.50 (WaveSpeed, 5 s minimum)	\$9.00	WaveSpeed extend only [backend unclear]
Wan 2.5	Undisclosed	\$0.40 (WaveSpeed)	\$7.20	WaveSpeed extend only [backend unclear]
LTX-2.3 (open weights)	22B	~0.14 (fal: ~0.036/s; +~\$0.07 if the input context is also billed)	~\$2.60	fal ltx-2.3-22b/extend (context 0–121 frames) · fal ltx-2.3-quality/extend (~0.053/s) · WaveSpeed (0.10/s) · Atlas [price looks wrong]
LTX-2.3 Pro	22B	— (rejects a 2 s input; needs ≥73 frames)	—	LTX API v1 [ends 10/26] · LTX API v2 · fal · Replicate
SkyReels-V 3 V2V	14B	\$0.20 (APIFree: \$0.04/s, 5 s minimum)	\$3.60	APIFree (Skywork's official channel; input-length limit not documented) · WaveSpeed [likely delisted]
MAGI-1 distilled	Not stated (likely 24B distill)	\$0.32 (fal: 96-frame minimum)	\$5.76	fal magi-distilled/extend (conditions on the whole prefix) · Sand AI official "magi" [~\$0.10/s, unverified] · fal magi [shut down]
Cosmos 3 Nano	16B (8B generator)	Free (rate-limited)	Free	NVIDIA build free trial (can condition on the last frames) · DeepInfra [ends 10/2]. Max 5 input frames
Cosmos 3 Super	64B (32B generator)	\$0.20 (DeepInfra)	\$3.60	DeepInfra only [ends 10/2–3]. No V2V option after 10/6
Cosmos-Pr edict2.5	2B	\$0.20 per video (fal; fixed 5.8 s)	\$3.60	fal (number of conditioning frames not documented) · WaveSpeed V2V [page broken]
Veo 3.1 / Fast / Lite	Undisclosed	\$1.40 / \$0.56 / \$0.21 (Vertex; extend output fixed at 7 s)	\$25.20 / \$10.08 / \$3.78	Vertex (input 1–30 s at 24 fps; arbitrary videos [unverified]; continues from the last 1 s of the input) · fal / AIML extend (probably Vertex underneath) · Gemini API / WaveSpeed / EachLabs [Veo-generated videos only]

*Not included because no provider offers true continuation of an uploaded video: Kling (all versions), Runway, Luma, MiniMax H3, grok-imagine-video-1.5.

I2V Options (not used for now: V2V only)

Model	Params	Cost / generation	Cost × 18	Where available (API)
Veo 3.1	Undisclosed	\$0.80 (Vertex)	\$14.40	Vertex · Gemini API (1.60, <i>audioon</i>) · <i>fal</i> · <i>Replicate</i> · <i>WaveSpeed</i> (0.48) · AIML (1.04) · <i>Kie</i> (1.25 per video)
Veo 3.1 Fast	Undisclosed	\$0.32 (Vertex)	\$5.76	Vertex · OpenRouter · Atlas · Gemini API (0.40) · <i>fal</i> · <i>Replicate</i> · <i>WaveSpeed</i> (0.24) · AIML · <i>Kie</i> (\$0.30 per video)
Veo 3.1 Lite	Undisclosed	\$0.12 (Vertex) [preview]	\$2.16	Vertex · <i>fal</i> · OpenRouter · Gemini API [preview] · <i>Replicate</i> · <i>WaveSpeed</i> · Atlas (\$0.20) · AIML · <i>Kie</i>
Gemini Omni 1.1 Flash	Undisclosed	~\$0.41 (Gemini API; audio always on)	~\$7.29	Gemini API · Vertex [preview] · <i>fal</i> · <i>WaveSpeed</i> · Runway · Atlas · AIML (0.53) · <i>Replicate</i> (0.15/s; duration can't be set) · <i>Kie</i> (\$0.315)
xAI grok-imagine-video	Undisclosed	\$0.28 (xAI)	\$5.08	xAI · <i>fal</i> · OpenRouter · <i>Replicate</i> / <i>WaveSpeed</i> / Atlas (\$0.20) · AIML · <i>Kie</i> · Poe / CometAPI [unverified]
xAI grok-imagine-video-1.5	Undisclosed	\$0.57 (xAI)	\$10.26	xAI · <i>fal</i> · <i>WaveSpeed</i> · OpenRouter · <i>Replicate</i> (0.32) · <i>Runway</i> (0.65) · AIML · <i>Kie</i> · Atlas
Seedance 2.5	Undisclosed	\$0.92 (BytePlus; audio included)	\$16.63	BytePlus · Volcengine · OpenRouter · Segmind · <i>Replicate</i> · Runware · AIML · EvoLink · <i>Kie</i> · PiAPI · <i>WaveSpeed</i> · <i>fal</i> (1.85) · <i>Runway</i> (1.20)
Seedance 2.0	Undisclosed	\$0.61 (BytePlus; audio included)	\$10.89	BytePlus · Volcengine · OpenRouter · Segmind · Runware · <i>Replicate</i> (0.72) · <i>EvoLink</i> · <i>PiAPI</i> · <i>Kie</i> · <i>Wave</i> (0.96) · <i>fal</i> (1.21) · <i>Runway</i> (1.44)

Seedance 2.0 Fast	Undisclosed	\$0.48 (BytePlus)	\$8.71	Same as 2.0 · Segmind (\$0.24)
Seedance 2.0 Mini	Undisclosed	\$0.30 (BytePlus)	\$5.44	Same as 2.0 · Segmind (\$0.15)
MiniMax H3	33B (open weights)	\$0.32 (MiniMax, 768P)	\$5.76	MiniMax official (intl / CN) · Replicate · OpenRouter · Runware · fal (0.30, 5s minimum) · Novita (0.28) · Kie (0.16) · CometAPI · Runway (0.40) · AIML (2K only) · WaveSpeed / PiAPI / Together (run the weights themselves)
MiniMax H3-Max	Undisclosed	\$0.40 (MiniMax; 5 s minimum)	\$7.20	MiniMax official · fal (\$0.20 promo until 9/30) · Runway · OpenRouter · Runware · Atlas · CometAPI · AIML
Hailuo 02	Undisclosed	\$0.28 (MiniMax; 6 s minimum) [legacy]	\$5.04	MiniMax [legacy] · fal · Replicate · Together · AIML · Runware · EachLabs
Kling 3.0	Undisclosed	\$0.34 (Kling official)	\$6.05	Kling official · fal · WaveSpeed · OpenRouter · Runware · Novita · Kie (0.28) · Atlas (0.28) · AIML · Replicate / Segmind (\$0.67) · PiAPI (5 s minimum) · EachLabs / CometAPI [unverified]
Kling 3.0 Omni	Undisclosed	\$0.34 (Kling official)	\$6.05	Kling official · fal · WaveSpeed · Runware · Atlas · PiAPI · Replicate (0.67) · Segmind (0.84)
Kling 3.0 Turbo	Undisclosed	\$0.45 (Kling official; audio always on)	\$8.06	Kling official · fal · WaveSpeed · Runware · Kie · Atlas · AIML
Kling 2.6	Undisclosed	\$0.21 (Kling official; 5 s minimum)	\$3.78	Kling official · WaveSpeed · PiAPI · Atlas · fal / Replicate (pro only, \$0.35) · Kie · AIML · Segmind

Runway Gen-4.5	Undisclosed	\$0.48 (Runway)	\$8.64	Runway official · OpenRouter · Replicate / Runware (5 s minimum, \$0.60) · CometAPI · Magnific
Runway Gen-4 Turbo	Undisclosed	\$0.20 (Runway)	\$3.60	Runway official · Runware · Replicate / WaveSpeed (5 s minimum) · AIML · Segmind · CometAPI
Luma Ray 3.2	Undisclosed	\$0.30 (Luma; fixed 5 s) [pre-GA]	\$5.40	Luma official · fal · Replicate · Runware · Atlas · WaveSpeed (1.00) · <i>Segmind</i> (1.25) · AIML [unverified]
Luma Ray 2	Undisclosed	\$0.40 (WaveSpeed; 5 s) [legacy]	\$7.20	WaveSpeed · Replicate (0.90) · <i>fal</i> (1.00) · AIML · PiAPI
Cosmos 3 Super	64B (32B generator)	\$0.20 (fal)	\$3.60	<i>fal</i> · WaveSpeed · Atlas (\$0.22) · DeepInfra [ends 10/2–3]
Cosmos 3 Nano	16B (8B generator)	Free (rate-limited)	Free	NVIDIA build free trial · DeepInfra [ends 10/2]
Cosmos-Predict2.5	2B	\$0.20 per video (<i>fal</i> ; fixed 5.8 s, ~7 min each)	\$3.60	<i>fal</i> · WaveSpeed (\$0.25) · JAI Portal [unverified]
Cosmos-Predict1	7B	—	—	NVIDIA [shut down]
MAGI-1	24B (distilled size not stated)	\$0.32 (<i>fal</i> distilled)	\$5.76	<i>fal</i> magi-distilled · WaveSpeed (24B, \$0.32) · Sand AI official [price unverified] · <i>fal</i> magi [shut down] · Atlas [delisted]
MAGI-2-preview	114B (MoE)	Undisclosed (fixed 10 s)	—	Sand AI [access by request only]
Wan 3.0	Undisclosed [beta]	\$0.40 (Alibaba)	\$7.20	Alibaba · fal · Replicate · Runway · Atlas (\$0.16) · Kie
Wan 2.7	Undisclosed	\$0.40 (Alibaba)	\$7.20	Alibaba · fal · Replicate · Atlas · Novita · Together
Wan 2.6 / 2.5	Undisclosed	\$0.50 (5 s minimum)	\$9.00	Alibaba · fal · Replicate

Wan 2.2	A14B (27B MoE, 14B active) / 5B	\$0.32 (fal A14B)	\$5.76	fal · APIFree (0.10, 5s) · <i>Alibabaflash</i> (0.14) · WaveSpeed (0.30, 5s) · <i>Replicate</i> (1 per video) · SiliconFlow (\$0.29 per video) · Segmind / Atlas [unverified]
LTX-2.5	22B	\$0.54 Fast / \$0.72 Pro (LTX API; 6 s minimum)	\$9.72 / \$12.96	LTX API · fal · WaveSpeed (\$0.50, 5 s) · <i>Replicate</i> [price looks wrong]
LTX-2.3	22B	\$0.18 (LTX API Fast; 6 s minimum)	\$3.24	LTX API · fal open 22B (~\$0.14) · fal Pro (1080p only) · <i>Replicate</i> (1080p) · WaveSpeed
HunyuanVideo 1.5	8.3B	\$0.20 (WaveSpeed; 5 s minimum)	\$3.60	WaveSpeed · fal (480p only) · Tencent Cloud (model version undisclosed)
SkyReels V4	Undisclosed	\$0.56 (WaveSpeed)	\$10.08	WaveSpeed (V3 has no first-frame I2V)
Kandinsky 5.0 Video Pro	19B (reported)	\$0.20 (512P, 5 s; no 720p tier)	\$3.60	fal · WaveSpeed · EachLabs

Appendix: VIMS

VIMS (View-Invariant Motion Similarity) measures whether the three camera-angle generations show the same 3D motion. It uses only the tracked ball centers, with no size or depth estimate: if the three videos show the same motion, the three lines of sight to the ball meet at one point in every frame.

Lines of sight. In frame t , view k (camera center C_k , intrinsics K_k , rotation R_k) sees the ball at pixel (u_{kt}, v_{kt}) , which gives the unit direction $\mathbf{d}_{kt} \propto R_k K_k^{-1} [u_{kt}, v_{kt}, 1]^{\top}$.

Consensus point. The point closest to the three lines in the least-squares sense: $\hat{X}_t = \left(\sum_k P_{kt} \right)^{-1} \sum_k P_{kt} C_k$, $\mathbf{d}_{kt} \cdot \hat{X}_t = 0$

Inconsistency. e_t is the root-mean-square distance in meters from $\hat{X}_t$ to the three lines. E is the mean of e_t over the generated frames after a 5-frame running median, which removes tracker glitches of 1 or 2 frames but keeps any disagreement that lasts 3 frames or more. Lower E means more consistent views.

Robustness steps. The image centroid of a sphere is not the projection of its center, so tracked centers are corrected in closed form before triangulation. Single-frame tracking jumps are removed. Every video is aligned so that frame 0 is the last conditioning frame, and videos whose camera moves are flagged.

Score. $\text{VIMS} = \exp \left(-\frac{\max(E - E_{\text{ceil}}, 0)}{E_{\text{ref}}} \right)$ where E_{ceil} is the E produced by tracker noise alone (0.3 px) and E_{ref} the E when one view lags by one frame, both computed from the scene's ground truth. VIMS is 1 when the views agree as closely as tracking noise allows and e^{-1} at a one-frame lag. For our camera-angle scene (generated part, frames 48 to 143): $E_{\text{ceil}} = 1.3$ mm and $E_{\text{ref}} = 14.7$ mm.

Validation. On Kubric renders tracked with SAM 2.1 plus sub-pixel refinement, three views of the same motion give $E = 0.1$ to 0.5 cm under blur and motion blur. A view showing a different motion (restitution changed, speed +5%, or a lag of 1 or 2 frames) is separated from same-motion triplets with AUROC 1.000, and the deviating view is named correctly in 98% of cases and never wrongly. On the dataset's three camera-angle clips the tracker error is about 0.1 px and $E = 0.25$ mm. The lines of sight should be at least about 40° apart (ours: 54° to 92°).